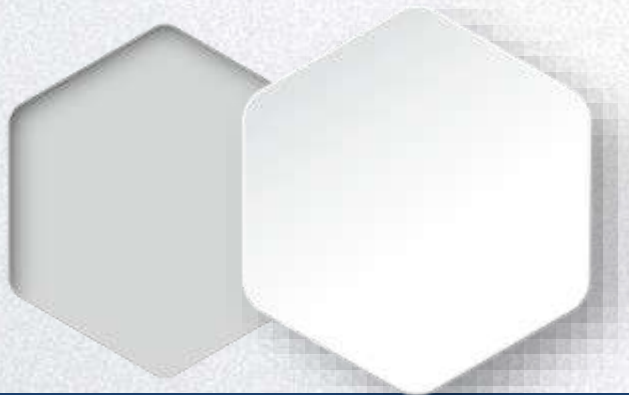




《人工智能与Python程序设计》——循环神经网络的原理和实现

如何对序列建模（语音，文字等）



人工智能与Python程序设计 教研组



提纲



人工智能与Python程序设计11-2
循环神经网络的原理和实现

- ☐ 循环神经网络的原理和结构
- ☐ 基于字符级别循环神经网络的文本分类
- 模型



如何使用神经网络处理自然语言文档?

- 自然语言是一个由字或词等符号 (token) 组成的序列
 - 长度可变
 - 顺序会对自然语言的语义产生影响
 - 词语之间具有依赖关系
- 自然语言处理要求我们使用能对**序列**进行建模的神经网络模型

不容易建模的
特点:

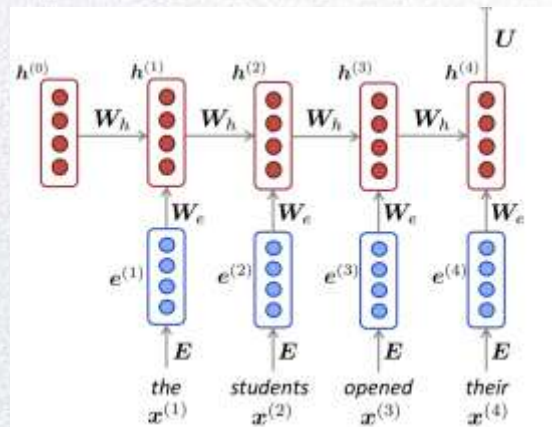
回顾：基于循环神经网络的语言模型

- 基于循环神经网络的语言模型 (Recurrent Neural Networks, RNN)

- RNN语言模型分析

- 优点

- 可以处理任意长度的输入
- 当前时间的计算可以利用之前的信息
- 模型大小并不随输入的长度变化



- 在这节课中我们将介绍如何使用PyTorch搭建一个循环神经网络

循环神经网络

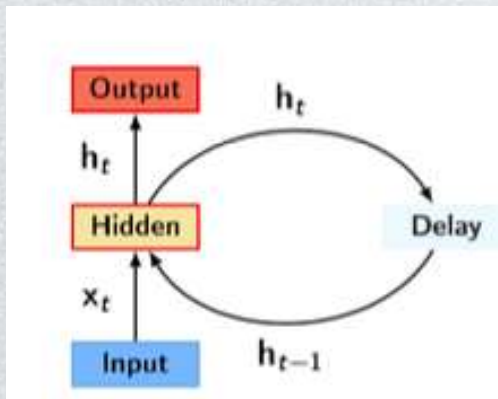
- 循环神经网络的结构

- 具有一组隐状态 (hidden state) 向量: h_t

隐状态向量 h_t 由当前输入 x_t 和前一个隐状态向量 h_{t-1}

在给定一个输入序列 $X = (x_1, x_2, \dots, x_t, \dots, x_n)$, “循环的” 更新 h_t

一个假设是之前的所有信息都背压缩到了隐状态 h_{t-1} 当中



$$h_t = \begin{cases} 0 & \text{if } t = 0 \\ f(h_{t-1}, x_t) & \text{otherwise} \end{cases}$$



循环神经网络

- 可以定义不同的更新隐状态向量的方法
- 一种简单的方法是：

$$h_t = f(h_{t-1} \cdot U + x_t \cdot W + b)$$

转化为1*k维的向量

1*d维的

- 其中：

变量	含义	维度(Shape)
h_t	t 时刻隐状态向量	(1, d)
x_t	t 时刻输入(查表后)向量	(1, k)
f	非线性映射	-
U	隐状态转移权重矩阵	(d, d)
W	输入到隐状态映射矩阵	(k, d)
b	隐状态向量偏置	(1, d)

使得两个变量的维度一样



使用pyTorch实现循环神经网络

- 使用pyTorch实现该函数:

$$h_t = f(h_{t-1} \cdot U + x_t \cdot W + b)$$

非线性层

```
class RNNCell(nn.Module):  
    def __init__(self, input_size, hidden_size):  
        super().__init__()  
  
        self.hidden_size = hidden_size  
        normalization_factor = hidden_size ** 0.5  
        self.W = nn.Parameter(torch.randn(input_size, hidden_size) / normalization_factor)  
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)  
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)  
        self.non_linear = nn.ReLU()  
  
    def forward(self, input, hidden=None):  
        if hidden is None:  
            hidden = torch.zeros(1, self.hidden_size)  
        hidden = hidden.matmul(self.U) + input.matmul(self.W) + self.b  
        hidden = self.non_linear(hidden)  
        return hidden
```



- 通过

```
rnn_cell = RNNCell(20, 10)
input_tensor = torch.randn(2, 1, 20)
hidden = None
for i in range(input_tensor.shape[0]):
    hidden = rnn_cell(input_tensor[i], hidden)
    print(i, hidden)
```

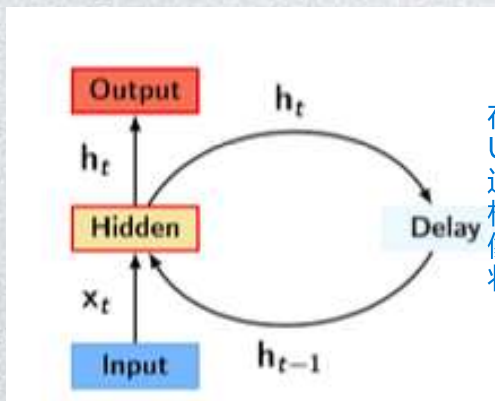
```
0 tensor([[0.7448, 1.4827, 3.0089, 3.0465, 2.3865, 1.1469, 0.0000, 0.0000, 0.0000,
          1.0028]], grad_fn=<ReluBackward0>)
1 tensor([[0.0000, 1.1701, 0.0000, 0.0000, 0.0000, 1.9843, 0.0000, 1.3233, 0.2943,
          0.0000]], grad_fn=<ReluBackward0>)
```

```
input_tensor = torch.randn(5, 1, 20)
hidden = None
for i in range(input_tensor.shape[0]):
    hidden = rnn_cell(input_tensor[i], hidden)
    print(i, hidden)
```

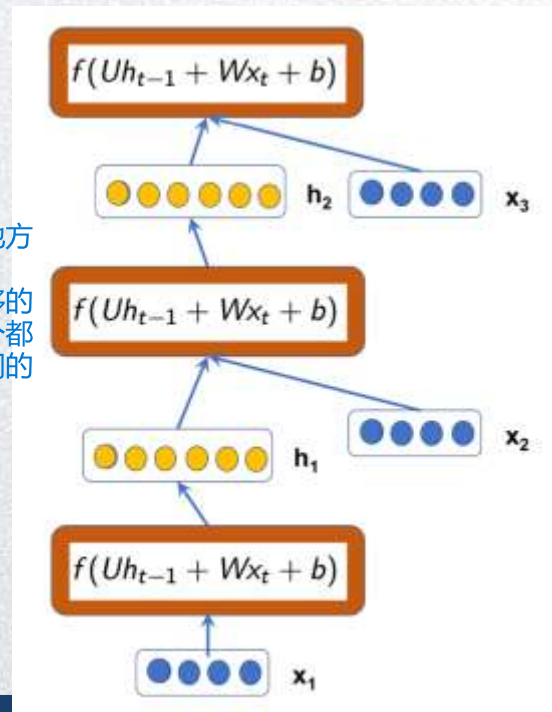
```
0 tensor([[0.0000, 0.0000, 0.4602, 1.6214, 1.8450, 0.0000, 1.2190, 0.0000, 0.3957,
          0.0000]], grad_fn=<ReluBackward0>)
1 tensor([[1.0400, 1.8433, 3.1282, 0.0000, 1.5801, 0.1586, 0.3469, 0.3237, 3.1358,
          0.0000]], grad_fn=<ReluBackward0>)
2 tensor([[0.0000, 0.6067, 0.0000, 0.0000, 0.3844, 0.2232, 0.0000, 0.0000, 2.5534,
          0.1269]], grad_fn=<ReluBackward0>)
3 tensor([[0.0000, 0.0000, 2.3848, 0.0000, 0.1908, 0.1591, 0.0000, 1.5654, 0.0000,
          0.0000]], grad_fn=<ReluBackward0>)
4 tensor([[1.1500, 1.8459, 1.8786, 1.2277, 1.3489, 0.0000, 0.0000, 0.0000, 0.0000,
          0.8068]], grad_fn=<ReluBackward0>)
```

使用pyTorch实现循环神经网络

- Pytorch会自动生成相应的计算图，支持自动梯度计算



在每一个处理的地方
UWb都是共享的：
这样可以识别足够的
相同趋势，每一个都
像是进入当前时间的
状态



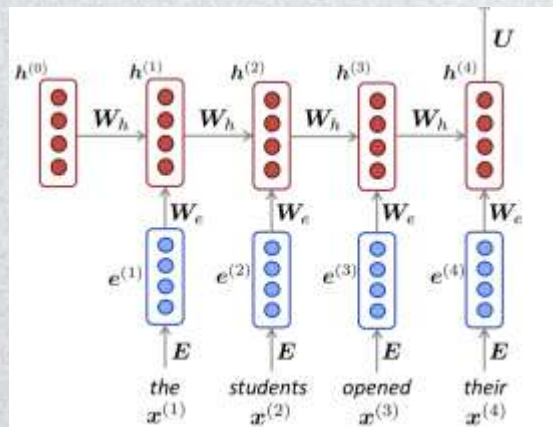
循环神经网络的问题是ON的，
只能依次去算，比较大的问题是
无法进行并行训练

训练的速度就非常缓慢

使用pyTorch实现循环神经网络

- 带输出层的循环神经网络：

$$o_t = h_t \cdot W_{out} + b_{out}$$



```
class RNNCellWithOutput(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()

        self.hidden_size = hidden_size
        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(input_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```



使用pyTorch实现循环神经网络

- 使用带输出层的循环神经网络

```
rnn_cell = RNNCellWithOutput(20, 10, 5)
input_tensor = torch.randn(2, 1, 20)
hidden = None
for i in range(input_tensor.shape[0]):
    output, hidden = rnn_cell(input_tensor[i], hidden)
    print(i)
    print('output', output)
    print('hidden', hidden)
```

```
0
output tensor([[ 0.6465, -0.8802, -0.6869,  0.2793, -0.6667]],
              grad_fn=<AddmmBackward>)
hidden tensor([[0.0000, 0.8727, 0.4114, 0.3919, 0.6481, 0.0000, 0.6365, 3.1681, 0.0000,
               0.0000]], grad_fn=<ReluBackward0>)
1
output tensor([[ 1.1495, -1.3209, -0.9556, -0.1129, -0.5748]],
              grad_fn=<AddmmBackward>)
hidden tensor([[0.3032, 0.0000, 0.7478, 0.0000, 2.3112, 0.0000, 0.0000, 1.4583, 0.0000,
               2.7281]], grad_fn=<ReluBackward0>)
```



提纲



人工智能与Python程序设计11-1
循环神经网络的原理和实现

- ☐ 循环神经网络的原理和结构
- ☐ 基于字符级别循环神经网络的文本分类
- 模型



使用pyTorch实现循环神经网络

- 使用pyTorch实现一个基于RNN的文本分类器

- 输入：人名
- 输出：这是哪种语言中常见的人名

计算机是没有智能的，它唯一能够识别的东西是张量

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取

模型训练

- 分类损失函数
- 损失函数优化和参数调优

模型测试

- 性能评价

数据准备



- 基于RNN的名字分类数据集：
 - 20,074行CSV文件
 - 每一行包含：人名，国名

```
1 Ahn,Korean
2 Baik,Korean
3 Bang,Korean
4 Byon,Korean
5 Cha,Korean
6 Chang,Korean
7 Chi,Korean
8 Chin,Korean
9 Cho,Korean
10 Choe,Korean
11 Choi,Korean
12 Chong,Korean
13 Chou,Korean
14 Chu,Korean
15 Chun,Korean
16 Chung,Korean
17 Chweh,Korean
18 Gil,Korean
19 Gu,Korean
20 Gwang,Korean
21 Ha,Korean
22 Han,Korean
23 Ho,Korean
24 Hong,Korean
25 Hung,Korean
26 Hwang,Korean
27 Hyun,Korean
28 Jang,Korean
29 Jeon,Korean
30 Jeong,Korean
```



如何表示输入？

- 目标：实现一个基于RNN的文本分类器

- 输入：人名

把每一个人名都看成字符串

- 输出：这是哪种语言中常见的人名

可以用循环神经网络计算

- 任务要求的输入：表示人名的字符串

- RNN模型的输入：长度可变的tensor

```
rnn_cell = RNNCell(20, 10)
input_tensor = torch.randn(2, 1, 20)
hidden = None
for i in range(input_tensor.shape[0]):
    hidden = rnn_cell(input_tensor[i], hidden)
    print(i, hidden)

0 tensor([[0.7448, 1.4827, 3.0089, 3.0465, 2.3865, 1.1469, 0.0000, 0.0000, 0.0000,
          1.0028]], grad_fn=<ReluBackward0>)
1 tensor([[0.0000, 1.1701, 0.0000, 0.0000, 0.0000, 1.9843, 0.0000, 1.3233, 0.2943,
          0.0000]], grad_fn=<ReluBackward0>)
```

- 如何将一个表示人名的字符串，转化为RNN可接受的输入呢？
 - Deng → D, e, n, g



- 第0步：读入数据

[illegible]



如何表示输入？

- 第1步：构建一个字典，将每个字符映射为一个整数，组成一个tensor
 - 首先使用**集合**，得到所有在人名中出现的字符，保存在一个**有序列表**中
 - 然后构建**字典**，将每个**字符**映射为这个字符在有序列表中出现的位置（**整数**）
 - 整数的取值范围为0~V-1，V是字符集合的大小
 - 最后使用该字典，将人名字符串，转变为一个**保存了整数的tensor**
 - 注意：name tensor long为一个shape为(字符串长度,1)的列向量，其中的元素为torch.long类型的整数，取值范围为0~V-1

```
[11]: # 将一个字符映射为一个整数tensor
import torch
index2char = sorted(set(''.join(names)))
char2index = {c:i for i, c in enumerate(index2char)}
name_tensor_long = torch.tensor([[char2index[c]] for c in 'Alice'], dtype=torch.long)

print('index2char', index2char[0:10])
print(char2index)
print(name_tensor_long)

index2char [' ', "'", 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
{' ': 0, "'": 1, 'A': 2, 'B': 3, 'C': 4, 'D': 5, 'E': 6, 'F': 7, 'G': 8, 'H': 9, 'I': 10, 'J': 11, 'K': 12, 'L': 13, 'M': 14, 'N': 15, 'O': 16, 'P': 17, 'Q': 18, 'R': 19, 'S': 20, 'T': 21, 'U': 22, 'V': 23, 'W': 24, 'X': 25, 'Y': 26, 'Z': 27, 'a': 28, 'b': 29, 'c': 30, 'd': 31, 'e': 32, 'f': 33, 'g': 34, 'h': 35, 'i': 36, 'j': 37, 'k': 38, 'l': 39, 'm': 40, 'n': 41, 'o': 42, 'p': 43, 'q': 44, 'r': 45, 's': 46, 't': 47, 'u': 48, 'v': 49, 'w': 50, 'x': 51, 'y': 52, 'z': 53}
tensor([[ 2],
        [39],
        [36],
        [30],
        [32]])
```



- Bob
- B, o, b
- 3, 42, 29

其实也没有太多的信息
所以要把字符先转化成ID, 然后把ID映射成一个词嵌入



如何表示输入？

机器学习的一系列模型，包括transformer等其实很核心的是数据表达

- 第2步：使用**nn.Embedding**层，将一个整数映射为一个多维浮点数组成的向量，使其能作为RNN模型的输入
 - 这种方法常被成为嵌入表示
 - 嵌入维度通常为几十维~几百维

```
[15]: # 使用nn.Embedding层，将用0~V-1整数表示词的文本转化成词嵌入表示矩阵
      from torch import nn
      embedding_dimension = 100
      embedding_layer = nn.Embedding(len(char2index), embedding_dimension)
      name_tensor_float = embedding_layer(name_tensor_long)
      print(name_tensor_float.shape)
      name_tensor_float

      torch.Size([5, 1, 100])

[15]: tensor([[[[-2.0978e+00,  1.7269e+00, -5.1127e-01, -2.0617e-01,  9.7511e-01,
          2.0260e+00,  1.1956e+00, -5.6362e-01, -8.4520e-01, -5.3024e-01,
          -5.2115e-01, -1.2494e+00, -2.7861e+00,  5.7239e-01,  6.2490e-01,
          -4.0992e-01,  1.2377e-01, -1.5592e-01,  1.9939e+00, -4.1127e-02,
          -4.1915e-01,  9.6891e-01,  7.7349e-01, -1.4265e-01, -1.7592e-01,
          2.5107e-01, -2.6341e-01, -1.0218e+00,  7.1431e-01,  8.2967e-01,
          -1.3828e+00, -3.6081e-01,  3.5180e+00, -5.4527e-02, -6.9404e-01,
          1.5808e+00, -2.9401e-01, -5.9635e-01,  4.7792e-01,  9.1426e-01,
          -7.4908e-01,  1.0147e+00, -1.0432e+00, -5.8009e-01,  1.2573e+00,
```



如何表示输出？

- 我们可以用类似的方法，将作为输出的国名映射为一个整数

处理类别

```
[17]: # 将作为输出的国名映射为一个整数
      index2country = sorted(set(countries))
      country2index = {c: i for i, c in enumerate(index2country)}
      print(country2index)

      country2index['Chinese']

{'Arabic': 0, 'Chinese': 1, 'Czech': 2, 'Dutch': 3, 'English': 4, 'French': 5, 'German': 6, 'Greek': 7, 'Irish': 8, 'Italian': 9, 'Japanese': 10, 'Korean': 11, 'Polish': 12, 'Portuguese': 13, 'Russian': 14, 'Scottish': 15, 'Spanish': 16, 'Vietnamese': 17}

[17]: 1
```

数据准备



- 将读取数据，生成字符到整数的映射，国名到整数的映射三个步骤整合到一个函数中，从文件读取模型训练和测试所需的输入X和输出y
 - 输入：X字符串数组
 - 输出：y整数数组

```
# 读取数据
def read_data(filename):
    X = list()
    y = list()

    with open(filename, 'r') as fin:
        for line in fin:
            # print(line)
            name, country = line.strip().split(',')
            X.append(name)
            y.append(country)

    # 构造字符到整数的映射
    index2char = sorted(set(''.join(X)))
    char2index = {c: i for i, c in enumerate(index2char)}
    X = np.array(X)

    # 构造country到整数的映射
    index2country = sorted(set(y))
    country2index = {c: i for i, c in enumerate(index2country)}
    y = np.array([[country2index[c]] for c in y])

    return X, y, index2char, char2index, index2country, country2index
```



数据准备

- 训练/测试集划分：
 - 随机打乱特征和预测目标顺序
 - 取打乱后的前split_ratio为训练集
 - 后1-split_ratio部分为测试集

```
def split_data(X, y, split_ratio=0.7):  
    # 训练测试划分  
    data_size = len(X)  
    shuffled_index = np.random.permutation(data_size)  
    X, y = X[shuffled_index], y[shuffled_index]  
    split_index = int(data_size * split_ratio)  
    X_train = X[:split_index]  
    y_train = y[:split_index]  
    X_test = X[split_index:]  
    y_test = y[split_index:]  
    return X_train, y_train, X_test, y_test
```



RNN模型构建

- 输入：
 - 使用nn.Embedding层将字符映射为嵌入向量

```
# 定义RNN
class RNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(embedding_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        input_embeddings = self.embeddings(input)
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input_embeddings.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```

RNN模型构建

- 计算隐状态向量:
 - 按照公式 $h_t = f(h_{t-1} \cdot U + x_t \cdot W + b)$, 计算隐状态向量
 - RNN的输入 x_t 为embedding层的输出

```
# 定义RNN
class RNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(embedding_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        input_embeddings = self.embeddings(input)
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input_embeddings.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```



RNN模型构建

- 输出：
 - 使用线性层得到一个维数和类别数量 (output_size=18)相同的向量

```
# 定义RNN
class RNN(nn.Module):
    def __init__(self, vocab_size, embedding_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        self.embeddings = nn.Embedding(vocab_size, embedding_size)

        normalization_factor = hidden_size ** 0.5
        self.W = nn.Parameter(torch.randn(embedding_size, hidden_size) / normalization_factor)
        self.U = nn.Parameter(torch.randn(hidden_size, hidden_size) / normalization_factor)
        self.b = nn.Parameter(torch.randn(1, hidden_size) / normalization_factor)
        self.non_linear = nn.ReLU()
        self.output = nn.Linear(hidden_size, output_size)

    def forward(self, input, hidden=None):
        input_embeddings = self.embeddings(input)
        if hidden is None:
            hidden = torch.zeros(1, self.hidden_size)
        hidden = hidden.matmul(self.U) + input_embeddings.matmul(self.W) + self.b
        hidden = self.non_linear(hidden)
        output = self.output(hidden)
        return output, hidden
```



RNN模型训练

- 定义损失函数:

- 多分类任务

- nn.CrossEntropyLoss()

- 注意: 该损失函数同时计算了softmax函数和交叉熵函数

$$\text{loss}(\mathbf{Z}, \mathbf{y}) = - \sum_{i=0}^{N-1} \log\left(\frac{e^{\mathbf{Z}[i, \mathbf{y}[i]]}}{\sum_j e^{\mathbf{Z}[i, j]}}\right)$$

```
# 定义RNN分类模型
class RNNClassifier():
    def __init__(self, char2index, output_dim, embedding_dim=128, hidden_dim=128):
        """
        创建模型和优化器, 初始RNN模型和优化器超参数
        """
        self.lr = 0.001
        self.epoches = 20

        self.num_chars = len(char2index) # 用于将字符映射到整数的字典, 其大小为字典大小
        self.char2index = char2index

        self.rnn = RNN(self.num_chars, embedding_dim, hidden_dim, output_dim)
        self.optimizer = torch.optim.SGD(self.rnn.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```



RNN模型训练

- 定义优化器：
 - torch.optim.SGD
 - 选择合适的学习率learning rate (lr) 和迭代次数
 - 学习率过低训练速度慢；学习率过高损失函数不下降
 - 迭代次数过少，损失函数未充分下降，模型训练不充分

```
# 定义RNN分类模型
class RNNClassifier():
    def __init__(self, char2index, output_dim, embedding_dim=128, hidden_dim=128):
        """
        创建模型和优化器，初始RNN模型和优化器超参数
        """
        self.lr = 0.001
        self.epoches = 20

        self.num_chars = len(char2index) # 用于将字符映射到整数的字典，其大小为字典大小
        self.char2index = char2index

        self.rnn = RNN(self.num_chars, embedding_dim, hidden_dim, output_dim)
        self.optimizer = torch.optim.SGD(self.rnn.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```



RNN模型训练

- 训练函数：外层循环
 - 多次 (epoches次) 遍历训练数据
 - 每次遍历前随机打乱数据的顺序

```
def train(self, X, y):  
    """  
    训练模型  
    输入：  
        X: 训练数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 训练数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回：  
        epoch_losses: 每个epoch的平均损失函数值  
    """  
    epoch_losses = []  
    data_size = X.shape[0]  
    for epoch in range(self.epoches):  
        losses = []  
  
        # 每轮训练前打乱数据的顺序  
        shuffled_index = np.random.permutation(data_size)  
        X, y = X[shuffled_index], y[shuffled_index]  
  
        for i in tqdm(range(data_size)):  
            # x, y = X[i], y[i]
```



RNN模型训练

- 训练函数：内层循环
 - 依次遍历每一个训练数据，计算损失函数，进行反向传播更新模型参数
 - tqdm：可用于显示进度条，迭代速度，预计完成时间等信息

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数，代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    #*循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)
```

```
100%|██████████| 14051/14051 [00:26<00:00, 533.24it/s]
0%|          | 51/14051 [00:00<00:27, 507.03it/s]
```



RNN模型训练

- 训练函数：内层循环
 - 将特征（人名字符串）转化为nn.Embeddings层需要的整数tensor

```
for i in tqdm(range(data.size)):  
    # x.shape = (length, 1)  
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)  
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类  
    y_true = torch.tensor(y[i], dtype=torch.long)  
  
    ##循环*神经网络!  
    hidden = None  
    for j in range(x.shape[0]):  
        output, hidden = self.rnn(x[j], hidden)  
    # output.shape = (1, 18)  
    loss = self.loss_function(output, y_true)  
  
    # 反向传播, 更新参数  
    self.optimizer.zero_grad()  
    loss.backward()  
    self.optimizer.step()  
  
    # 记录loss  
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 循环计算更新隐状态向量和输出向量

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    ##循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 10)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 利用最后一次得到的输出向量output和预测目标y，计算损失函数

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # 循环神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 模型梯度清零

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 调用backward计算梯度

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 利用优化器更新模型参数

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：内层循环
 - 记录loss，保存到losses列表中

```
for i in tqdm(range(data_size)):
    # x.shape = (length, 1)
    x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)
    # y_true.shape = (1, ), y 是一个{0, 1, ..., 17}范围内的整数, 代表x属于的分类
    y_true = torch.tensor(y[i], dtype=torch.long)

    # *循环*神经网络!
    hidden = None
    for j in range(x.shape[0]):
        output, hidden = self.rnn(x[j], hidden)
    # output.shape = (1, 18)
    loss = self.loss_function(output, y_true)

    # 反向传播, 更新参数
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    # 记录loss
    losses.append(loss.item())
```



RNN模型训练

- 训练函数：外层循环
 - 利用losses列表中记录的loss
 - 计算每次遍历数据集（每个epoch）的平均loss

```
self.optimizer.step()

# 记录loss
losses.append(loss.item())

# 计算每个epoch的平均loss并输出
epoch_losses.append(sum(losses) / data_size)
print("epoch: {}, avg loss is: {}".format(epoch, epoch_losses[-1]))

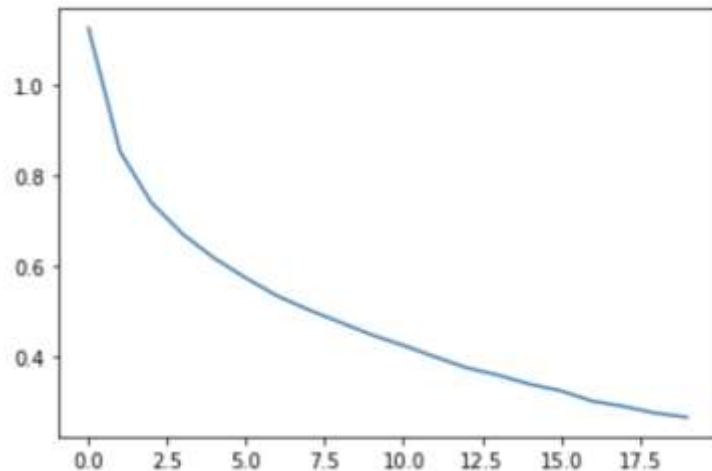
return epoch_losses
```

RNN模型训练

- 损失函数变化曲线：
 - 观察训练过程中损失函数的变化情况是一个调试机器学习和深度学习模型的重要技巧
 - 损失函数下降说明：
 - 模型能够拟合训练数据
 - 模型实现基本正确
 - 模型学习率设置合适

```
# 绘制损失函数变化曲线
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

plt.figure()
plt.plot(losses)
plt.show()
```



模型测试

- 基于训练好的模型进行分类：
 - 将特征（人名字符串）转化为nn.Embeddings层需要的整数tensor

```
def test(self, X, y):  
    """  
    用训练好的模型在测试集上进行测试  
    输入：  
        X: 测试数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 测试数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回：  
        y_pred: 测试数据的预测分类结果  
        accuracy: 分类准确率  
    """  
    data_size = X.shape[0]  
    y_pred = torch.empty(y.shape).long() # 预测分类与真实分类的形状相同  
    for i in range(data_size):  
        x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)  
  
        # *循环*神经网络!  
        hidden = None  
        for j in range(x.shape[0]):  
            output, hidden = self.rnn(x[j], hidden)  
        # output.shape = (1, 18)  
        top_n, top_i = output.topk(1)  
        y_pred[i] = top_i[0]
```

模型测试

- 基于训练好的模型进行分类：
 - 循环计算更新隐状态向量和输出向量

```
def test(self, X, y):  
    """  
    用训练好的模型在测试集上进行测试  
    输入:  
        X: 测试数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 测试数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回:  
        y_pred: 测试数据的预测分类结果  
        accuracy: 分类准确率  
    """  
    data_size = X.shape[0]  
    y_pred = torch.empty(y.shape).long() # 预测分类与真实分类的形状相同  
    for i in range(data_size):  
        x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)  
  
        # *循环*神经网络!  
        hidden = None  
        for j in range(x.shape[0]):  
            output, hidden = self.rnn(x[j], hidden)  
        # output.shape = (1, 18)  
        top_n, top_i = output.topk(1)  
        y_pred[i] = top_i[0]
```

模型测试

- 基于训练好的模型进行分类：
 - 使用topk函数，找出output中哪一维的值最大，得到模型预测结果

```
def test(self, X, y):  
    """  
    用训练好的模型在测试集上进行测试  
    输入：  
        X: 测试数据, (N, ), 每个元素是包含名字信息的字符串  
        y: 测试数据的真实分类, (N, 1), 每个元素是名字所属的类别  
    返回：  
        y_pred: 测试数据的预测分类结果  
        accuracy: 分类准确率  
    """  
    data_size = X.shape[0]  
    y_pred = torch.empty(y.shape).long() # 预测分类与真实分类的形状相同  
    for i in range(data_size):  
        x = torch.tensor([[self.char2index[c]] for c in X[i]], dtype=torch.long)  
  
        # *循环*神经网络!  
        hidden = None  
        for j in range(x.shape[0]):  
            output, hidden = self.rnn(x[j], hidden)  
  
            # output.shape = (1, 18)  
            top_n, top_i = output.topk(1)  
            y_pred[i] = top_i[0]
```

模型测试

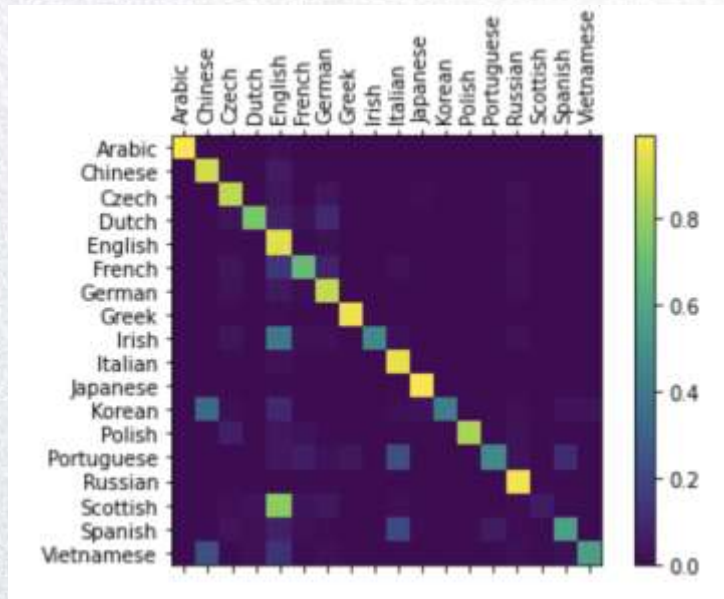
- 绘制混淆矩阵
 - 常用的展示多分类预测结果的方式
 - 矩阵第i行第j类表示把第i类结果分为第j类的概率
 - 在训练集上的准确率和混淆矩阵

如何写混淆矩阵的代码

```
[14]: # 在训练集上进行测试 在测试值上进行训练
y_pred, accuracy = classifier.test(X_train, y_train)
print("Accuracy on training set: {}".format(accuracy))

# 绘制混淆矩阵
plot_confusion_matrix(y_pred, y_train, country2index)
```

Accuracy on training set: 0.9312504529953003



模型测试

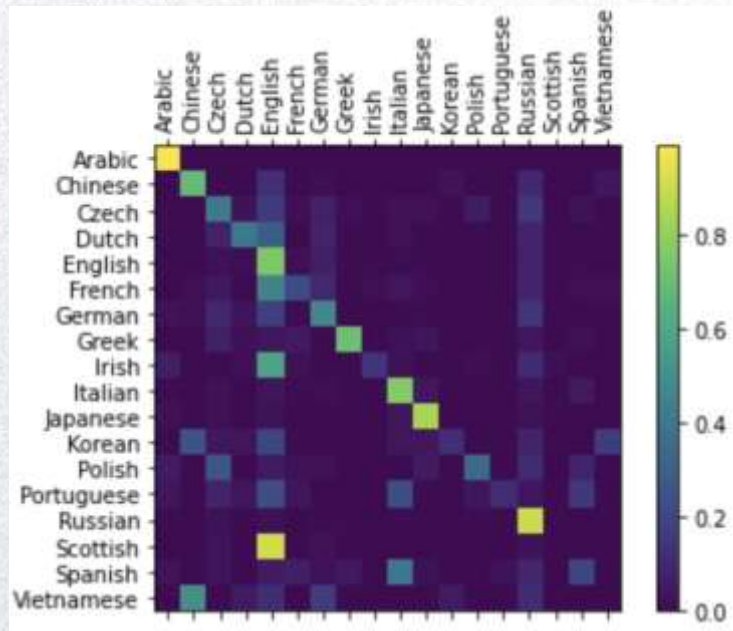
- 绘制混淆矩阵
 - 在测试集上的准确率和混淆矩阵

```
# 在测试集上进行测试
y_pred, accuracy = classifier.test(X_test, y_test)
print("Accuracy on test set: {}".format(accuracy))

# 绘制混淆矩阵
plot_confusion_matrix(y_pred, y_test, country2index)
```

Accuracy on test set: 0.7909679412841797

- 思考：
 - 为什么测试集上的正确率低于训练集？
 - 为什么Scottish会被错误分类为English？



总结



- 循环神经网络 (RNN)

- 能够对序列进行建模
- 使用pyTorch实现RNN
- 将文本表示为tensor
 - 整数序列
 - 词嵌入
- 实现基于RNN的文本分类器

- 下节课

- 更为复杂的循环神经网络 (LSTM、GRU)
- 使用pytorch实现基于RNN的语言模型和语言生成模型

1. 为什么要用循环神经网络建模序列，循环体现在哪里
2. 无论多复杂的图像、文字，都是tensor最后转化成分类问题



谢谢！



提纲



人工智能与Python程序设计11-2
循环神经网络的原理和实现

● □ 附录：word2vec简介



文本表示：词嵌入表示

- 我们除了可以得到字符的嵌入表示，我们也可以将词作为基本单元，利用词嵌入表示来表示文本
- 将一个词用一个固定维度（如100维）的稠密向量表示
 - 解决了占用空间和随词表规模线性增长的问题
 - 可以使用两个向量之间的内积或者余弦相似度来建模词之间的相似性
- 这种方法被称为词嵌入表示（word embedding）



文本表示：词嵌入表示

- 在pytorch中实现词嵌入表示：
 - 将独热表示矩阵与一个词嵌入权重矩阵相乘

```
[58]: # 词嵌入表示
      embedding_dimension = 100
      W = torch.randn(len(word2index), embedding_dimension)
      words_e = words_t @ W
      print(words_e.shape)
      words_e
      torch.Size([265, 100])
[58]: tensor([[ -0.3008,  0.4626,  0.3038, ...,  0.7862,  1.8939,  0.9709],
              [-1.1200, -0.7194,  1.4683, ...,  1.1613, -0.4520,  0.9987],
              [-0.5518, -0.0938, -0.1916, ...,  0.1136,  1.8078,  0.2906],
              ...,
              [ 2.3340,  0.6370,  0.4319, ...,  0.5893,  0.5294,  1.1769],
              [ 1.2955, -0.3759, -0.2123, ...,  0.5031,  0.5392,  0.9897],
              [ 0.9561,  0.3972, -0.8566, ...,  0.7499,  0.5519, -0.3844]])
```



文本表示：词嵌入表示

- 在pytorch中实现词嵌入表示：
 - 将独热表示矩阵与一个词嵌入权重矩阵相乘

```
[14]: # 词嵌入表示
      embedding_dimension = 100
      W = torch.randn(len(word2index), embedding_dimension, requires_grad=True)
      words_e = words_t @ W
      # words_e = torch.matmul(words_t, W)
      print(words_e.shape)
      words_e

      torch.Size([265, 100])
[14]: tensor([[ 0.0044, -0.3123,  1.0312, ..., -0.5518, -1.7435,  0.6691],
              [ 0.6764,  0.0131, -1.0707, ...,  1.1094,  1.0055, -0.1110],
              [ 0.9358, -1.7289, -1.3182, ...,  1.1275, -1.7838,  0.8176],
              ...,
              [-0.0469,  0.5683,  0.2476, ..., -0.5243,  0.9030,  1.0602],
              [-0.2331, -1.6221,  0.6599, ..., -0.0565,  1.0998, -1.1338],
              [ 0.1702, -0.9547, -1.1760, ...,  0.7208,  0.8150,  0.0871]],
      grad_fn=<MmBackward>)
```



文本表示：词嵌入表示

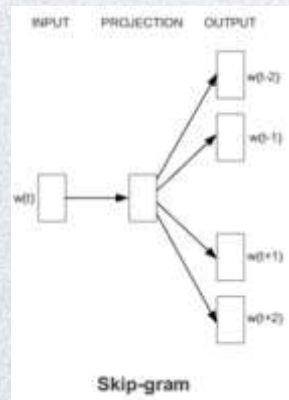
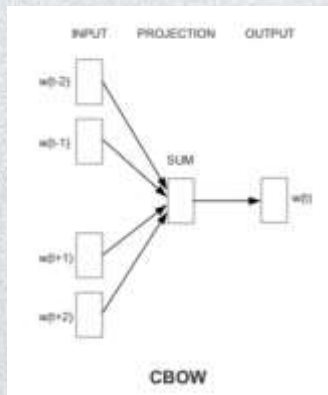
- 在pytorch中实现词嵌入表示：
 - 使用nn.Embedding层
 - 输入Tensor元素：用0~V-1的整数表示一个词的list

```
[65]: # 使用nn.Embedding层, 将用0~V-1整数表示词的文本转化成词嵌入表示矩阵
      from torch import nn
      embedding_layer = nn.Embedding(len(word2index), embedding_dimension)
      words_e = embedding_layer(torch.tensor(words_n))
      print(words_e.shape)
      words_e

      torch.Size([265, 100])
[65]: tensor([[ 0.6246, -0.4857,  0.1457, ...,  0.7153, -1.4111, -0.9076],
              [-0.6180, -0.4845,  0.2359, ..., -0.1283,  0.1288,  0.5852],
              [ 0.9348, -0.5666,  1.2014, ...,  0.9619,  0.0959,  2.5846],
              ...,
              [ 0.1577,  0.6339,  0.9073, ...,  0.4772, -1.3695,  1.7395],
              [ 3.9150, -0.2692,  0.0609, ...,  1.5839,  0.7613, -0.0155],
              [ 0.2566, -1.1048, -1.4751, ..., -1.1866,  0.2993,  0.4209]],
              grad_fn=<EmbeddingBackward>)
```

文本表示：词嵌入表示

- 如何训练词嵌入表示：
 - 针对具体任务直接端到端的优化W矩阵或者embedding_layer参数
 - 在大规模文本上通过训练语言模型的方法预训练词嵌入表示
 - 自监督学习
 - word2vec



文本表示：词嵌入表示

- 如何更好地表示词的语义？
- 分布式语义（distributional semantics）假说：
 - 一个词的含义由经常在改词附近的其他词所决定
 - “You shall know a word by the company it keeps”
J. R. Firth 1957
- 将一个词出现在文本时，其周围固定大小窗口内同时出现的其他词作为该词的上下文环境（context）
- 我们可以用一个词出现的上下文环境来构建其表示



...government debt problems turning into banking crises as happened in 2009...

...saying that Europe needs unified banking regulation to replace the hodgepodge...

...India has just given its banking system a shot in the arm...



文本表示：词嵌入表示

- Word2vec (Mikolov et al. 2013)

Distributed representations of words and phrases and their compositionality

24131

2013

T Mikolov, I Sutskever, K Chen, GS Corrado, J Dean
Neural information processing systems

Efficient estimation of word representations in vector space

19130

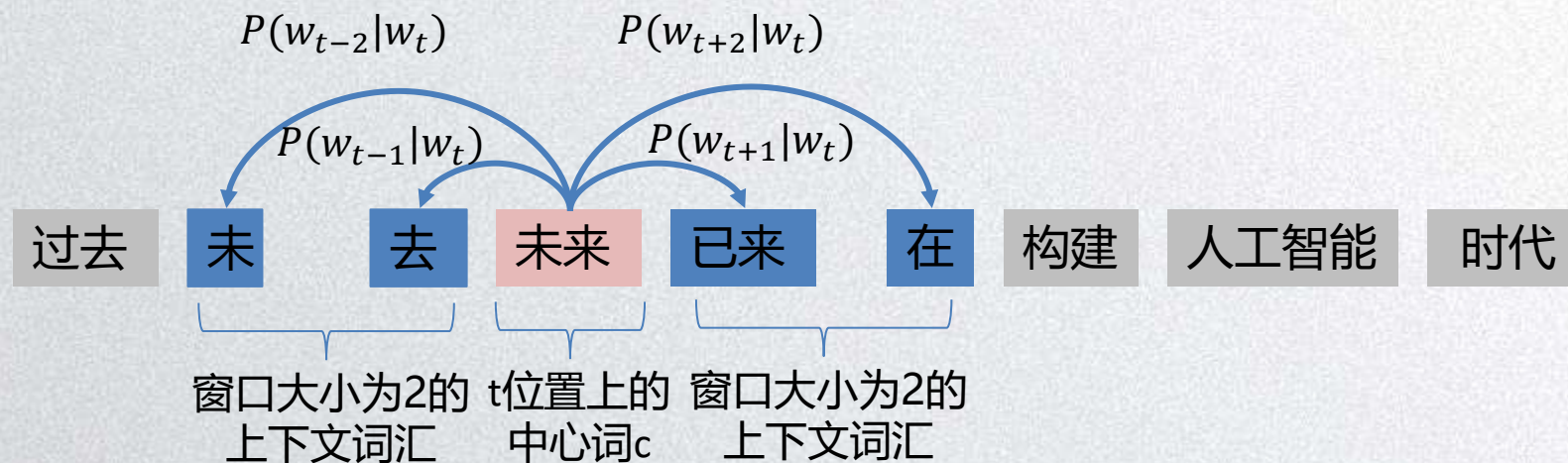
2013

T Mikolov, K Chen, G Corrado, J Dean
arXiv preprint arXiv:1301.3781

- 基本思路：
 - 收集大规模语料
 - 将每个词用词嵌入向量表示
 - 对于文本中的每个位置，找到一个中心词c和中心词对应的上下文中的词o
 - 用c和o的词嵌入向量的相似度来计算c和o每次同时出现的概率
 - 通过调整词嵌入向量最大化上述概率

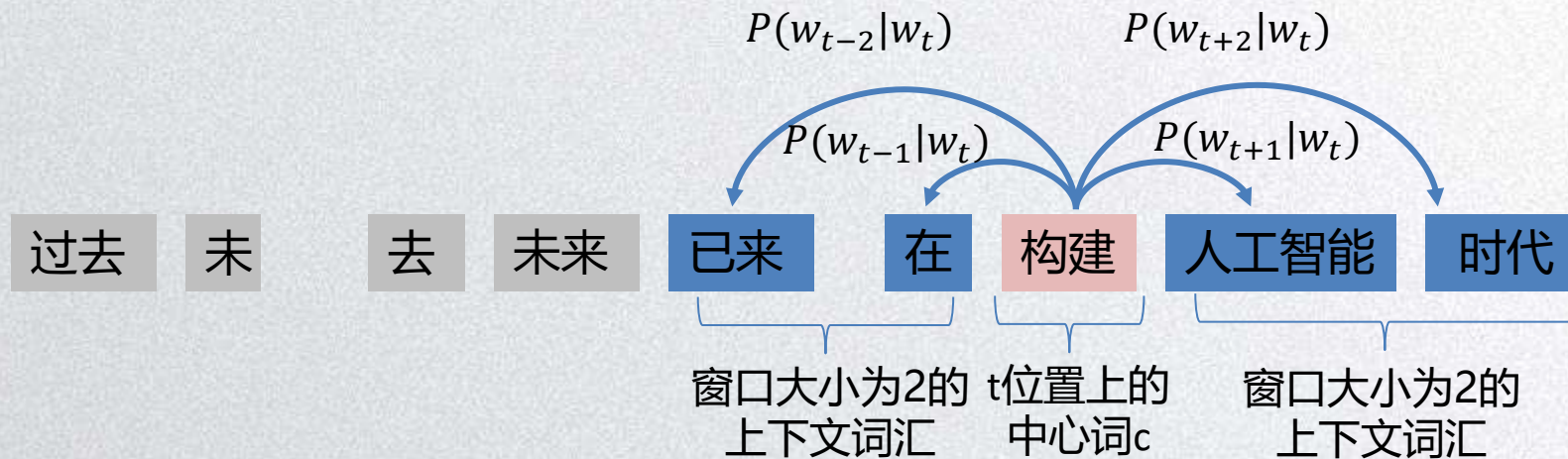
文本表示：词嵌入表示

- Word2vec的训练 (Skip-gram) :
 - 上下文的构建
 - $t=3$



文本表示：词嵌入表示

- Word2vec的训练 (Skip-gram) :
 - 上下文的构建
 - $t=6$





文本表示：词嵌入表示

- Word2vec的训练 (Skip-gram) :

- 对于文本中的每个位置 t ，给定当前中心词 w_t 预测在大小为 m 的窗口中出现的上下文词汇 w_{t+j}

- 似然函数：

$$L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w_{t+j} | w_t; \theta)$$

- 损失函数：负对数似然函数：

$$J(\theta) = - \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log(P(w_{t+j} | w_t; \theta))$$

- θ 表示模型参数，如 W 矩阵或embedding layer的参数

- 通过随机梯度下降算法，使负对数似然函数不断减小

文本表示：词嵌入表示

- Word2vec的训练 (Skip-gram) :
 - $P(w_{t+j}|w_t; \theta)$ 的计算
 - 一个词使用两个词嵌入向量表示
 - v_w : 作为中心词时使用
 - u_w : 作为上下文词汇时使用
 - 方便计算, 最后输出两个词嵌入向量的平均值作为词的表示

$$P(o|c) = \text{softmax}(x_o) = \frac{\exp(x_o)}{\sum_{w \in V} \exp(x_w)} = p_o$$
$$x_w = u_w^T v_c$$

- 即用softmax函数将中心词和上下文词对应词嵌入向量的内积映射为一个上下文词的概率分布



文本表示：词嵌入表示

- 可以使用训练好的词嵌入表示：
 - 英文：
 - torchtext库中提供了训练好的GloVe向量
 - 安装方法：pip install torchtext
 - 使用方法：调用构造函数后会自动下载相应的GloVe词向量

```
[*]: # 英文
vec = torchtext.vocab.GloVe(name='6B', dim=50)

.vector_cache/glove.6B.zip: 8% | 68.0M/862M [1:22:58<10:48:59,
20.4kB/s]
```

- 中文：
 - <https://github.com/Embedding/Chinese-Word-Vectors>
 - 提供了在不同数据集上用不同方法训练的词向量